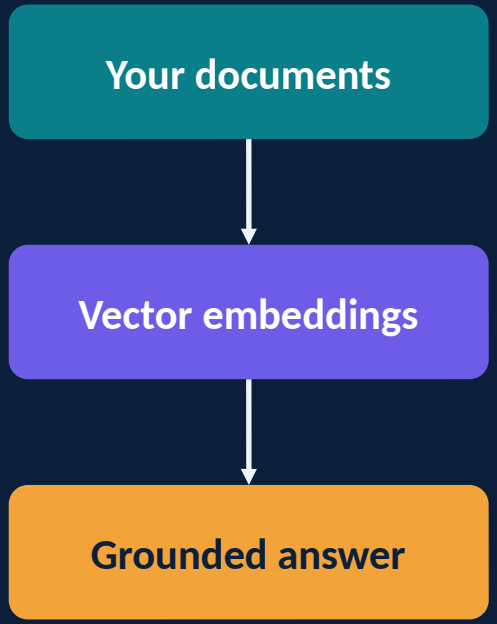
Agent Memory, Knowledge Systems & RAG

Module 6 | M.Tech practical lecture | 2 hours, with a live coding lab



Who am I?



Shubham Kumar Sahu

goodhacker.in

github.com/sksdbest

PhD Scholar, IIT Bhilai

Under Dr. Subidh Ali

Cyber Security Consultant

Central Business Solutions Inc.

Contact

skumars@iitbhilai.ac.in

Plan for the next 2 hours

A 25 min

Foundations

- Retrieval in AI systems
- Vector embeddings
- Semantic search

B 35 min

Indexing & RAG

- Document indexing
- RAG architecture
- RAG vs SLM, evaluation

C 30 min

Hands-on lab

- Clone and run our repo
- Code walkthrough
- Live demo, experiments

D 20 min

Agents & memory

- Agent knowledge integration
- Memory architectures
- Knowledge compression

Plus: 5 min introduction and 5 min wrap-up / Q&A

By the end, you can...

- Explain how retrieval grounds an LLM
- Compute and compare embedding similarity
- Design an indexing and chunking strategy
- Build and run a hybrid RAG pipeline
- Choose between RAG, SLMs and fine-tuning
- Design agent memory with compression

A

25 min

Foundations

Retrieval, embeddings and semantic search

- Retrieval in AI systems
- Vector embeddings and similarity
- Semantic search and vector indexes

Why LLMs need external knowledge

Stale knowledge

Training data has a cutoff. The model does not know last week's CVE or this semester's syllabus.

Hallucination

When facts are missing, models can still produce fluent but unsupported answers.

Private data

Your lab reports, client documents and code were never part of training.

Retrieval = give the model the right evidence at question time

Retrieval in AI systems

Type	How it matches	Best for	Examples
Keyword	Exact term overlap (TF-IDF, BM25)	IDs, error codes, names	grep, Elasticsearch
Semantic	Similarity of meaning (embeddings)	Paraphrases, questions	FAISS, Chroma
Hybrid	Both rankings, fused	Production default	Our lab code
Structured	Queries over tables or graphs	Facts, aggregates	Text-to-SQL, Neo4j
Tool / web	Live external sources	Fresh information	Search APIs, agents

Classical retrieval: TF-IDF and BM25

TF-IDF

```
tfidf(t,d) = tf(t,d) x log(N / df(t))
```

- tf: how often the term appears in the document
- df: how many documents contain the term
- Rare terms get more weight

BM25

```
score(q,d) = SUM IDF(t) *
   f (k1+1)
  ----------------------------
   f + k1 (1 - b + b |d|/avgdl)
```

- k1: term frequency saturates
- b: penalises long documents

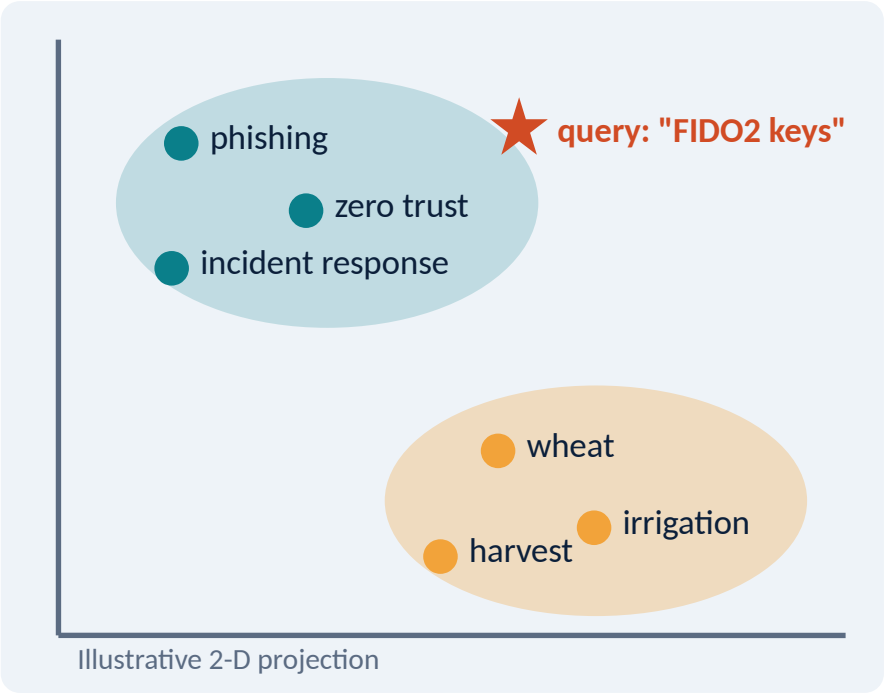
Limitation: no understanding of meaning. "car" does not match "automobile"

Vector embeddings

Text to vector: a model maps a sentence to a point in R^d (d = 384 for MiniLM)

Meaning to geometry: similar meaning means nearby points

No shared words needed: "patch the server" is near "apply a security update"



Illustrative 2-D projection

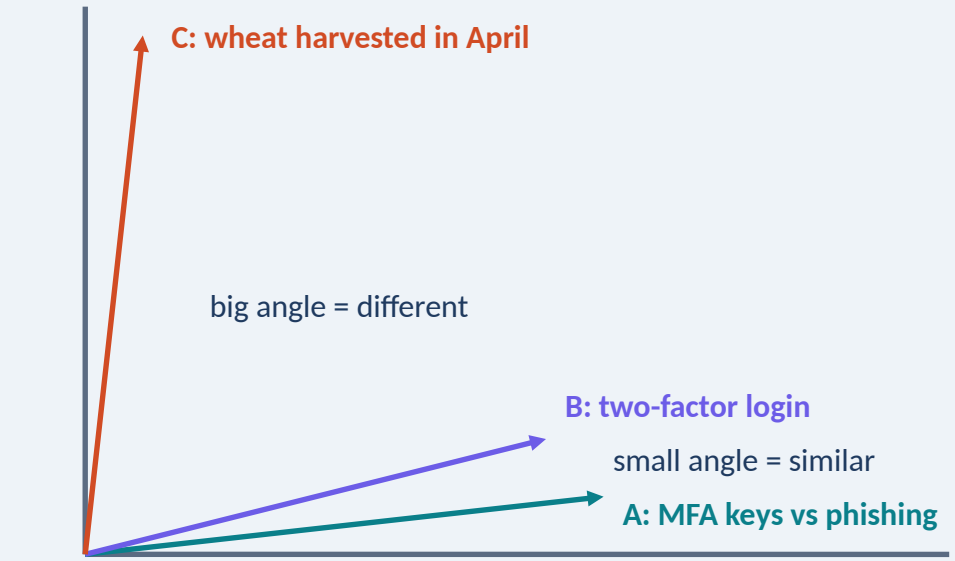
Where embeddings come from



TF-IDF / BM25	Word2Vec, 2013	BERT, 2018	Sentence-BERT, 2019	Modern models, 2023+
Sparse counts, no meaning	One dense vector per word	Vectors depend on context	Whole-sentence embeddings	Multilingual, long inputs, API or open

Contrastive training: pull matching pairs together, push non-matches apart

How similarity is measured



Horizontal axis = how much about security. Vertical = how much about farming.

- Each sentence becomes an arrow (a list of numbers)
- Similar meaning: arrows point the same way
- Score = how well the two arrows line up

1.0 same direction
0.0 unrelated (90 degrees)
-1.0 opposite

Cosine similarity, step by step

STEP 1

Multiply and add

```
A = (0.9, 0.1)
B = (0.8, 0.2)
0.9 x 0.8 + 0.1 x 0.2
= 0.74
```

STEP 2

Find each length

```
|A| = sqrt(0.81 + 0.01)
    = 0.91
|B| = sqrt(0.64 + 0.04)
    = 0.82
```

STEP 3

Divide

```
0.74 / (0.91 x 0.82)
= 0.74 / 0.75
= 0.99
```

A vs B = 0.99 similar

A vs C = 0.22 different

Real embeddings work the same way, with 384 numbers instead of 2.

Three ways to score similarity

Cosine

The angle between the arrows. Ignores how long they are.

Higher = more similar

```
a.b / (|a| x |b|)
```

Dot product

Multiply matching numbers, then add. Same as cosine when arrows have length 1 (our lab).

Higher = more similar

```
a1*b1 + a2*b2 + ...
```

Euclidean distance

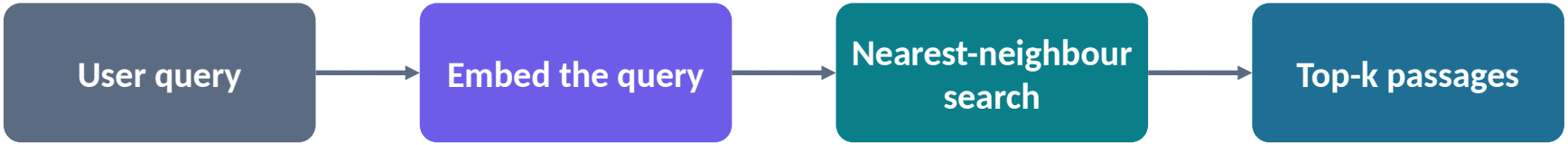
The straight-line distance between the two arrow tips.

Lower = more similar

```
sqrt( (a1-b1)^2
    + (a2-b2)^2 )
```

For text, cosine is the default. Our lab normalises vectors, so dot product = cosine.

Semantic search pipeline



Query: "How do I stop credential theft emails?"

Keyword search

No overlap with the words in "Phishing Defense", so it is missed.

Semantic search

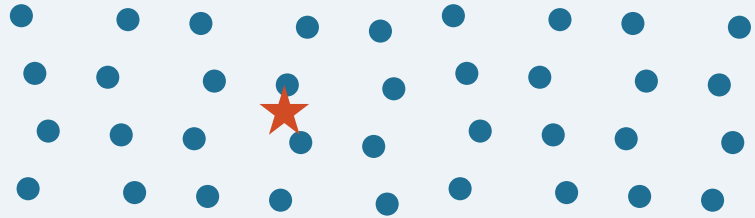
Meaning is close to the phishing section, so it ranks first.

ANN: Approximate Nearest Neighbour

Nearest neighbour: the stored vectors closest to the query (our top-k).

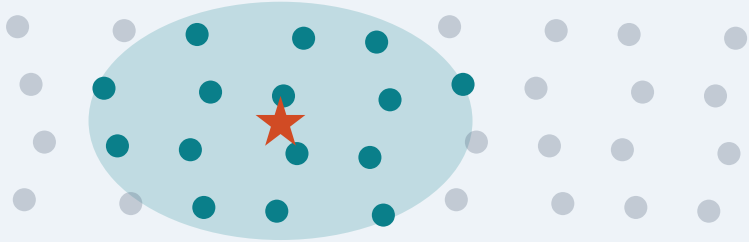
Approximate: use shortcuts, so a very close vector may rarely be missed.

Why: exact search over 1M vectors means 1M comparisons per question.



Exact search

Compare the query (star) with every vector.



ANN search

Jump to the nearby region, compare only those.

Three common ANN shortcuts

HNSW

Hierarchical Navigable Small World graph. Like a road map: highways first, then local streets.

IVF

Inverted File index. Like library sections: search only the groups closest to the query.

PQ

Product Quantisation. Like a zip file for vectors: much smaller, but slightly blurrier.

Trade-off: faster and smaller, but recall (share of true neighbours found) drops slightly

What is a vector database?

Stores each vector with its text and metadata

Builds the ANN index for you

Answers "top-k closest to this query", with filters

Option	In one line
Chroma	Easy to start, runs on your laptop
FAISS	Fast search library from Meta
pgvector	Adds vector search to PostgreSQL
Qdrant, Milvus	Dedicated servers built for scale

Our lab has few chunks, so plain matrix maths is enough. Use a vector DB at scale.

Check your understanding

1. A user searches "ERR_4021". Keyword or semantic search first? Why?

2. Two vectors have cosine 0.99 but very different lengths. Are they "similar"? When does that matter?

3. IVF searches fewer cells to go faster. What happens to recall, and why?

B

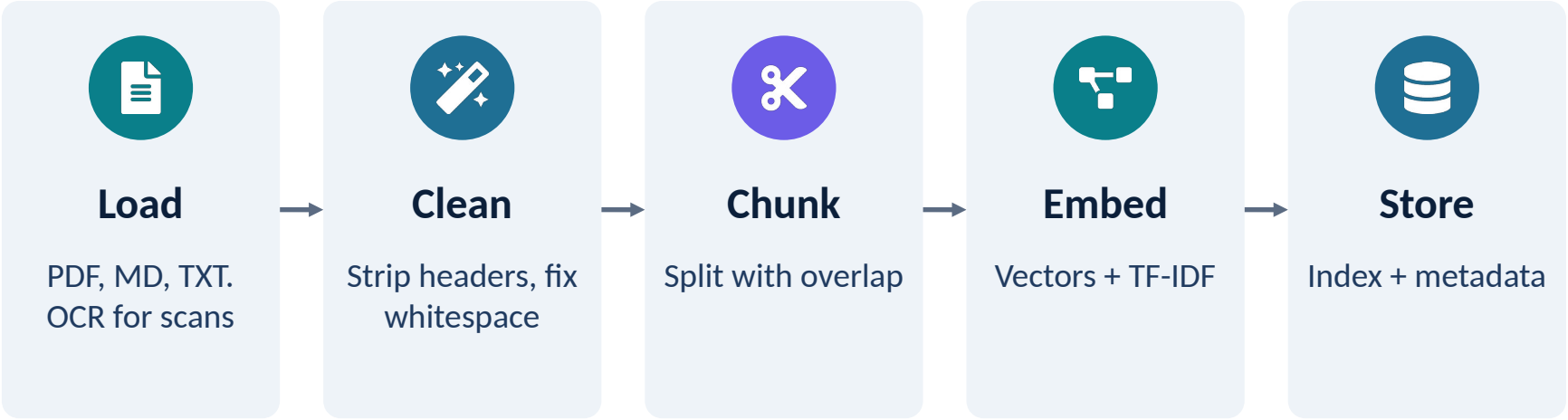
35 min

Indexing and RAG

From documents to grounded answers

- Document indexing and chunking
- RAG architecture, hybrid search, prompting
- RAG vs SLM vs fine-tuning
- Evaluation, failure modes and security

Document indexing pipeline




```
load_chunks()  ->  chunk_text()  ->  Index(chunks)  ->  save()
```

Chunking strategies

Strategy	How it works	Strength	Weakness
Fixed window	N words plus overlap	Simple, predictable	Cuts mid-idea
Structural	Split by headings and paragraphs, then size	Keeps sections intact (our lab)	Needs clean structure
Sentence / semantic	Break where meaning shifts	Coherent chunks	Slower, needs a model
Parent-child	Retrieve small, return large	Precise plus context	More complex index

Small chunks: precise, little context. Large chunks: more context, diluted signal.

Index structures

Inverted index

term to list of documents. Powers keyword search and BM25.

Vector index

ANN structure over embeddings. Powers semantic search.

Metadata filters

Source, date, author, access level. Filter before you rank.

```
wheat      -> [agro.txt#0]
phishing   -> [cybersecurity.md#2]
incident   -> [cybersecurity.md#1]
```

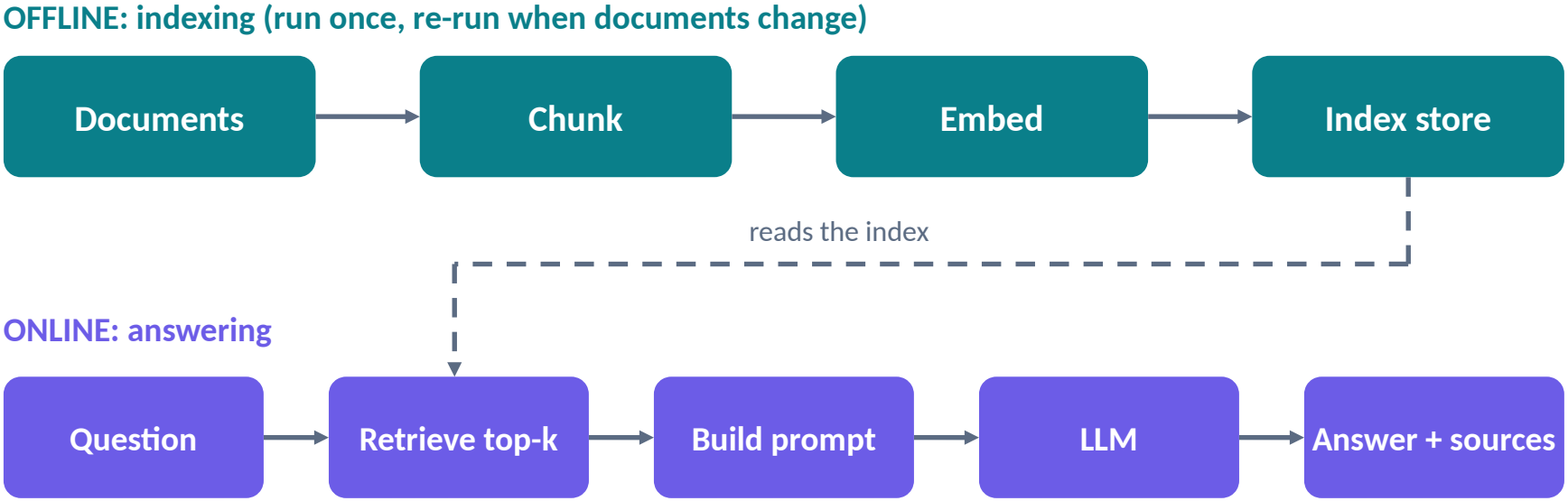
What is RAG?



Fresh | Grounded | Private | Auditable

RAG (Lewis et al., 2020): condition a generator on retrieved passages, not only on what is stored in its weights.

RAG architecture, end to end



Everything the LLM sees is chosen by retrieval. Wrong passages in means a wrong answer out.

Hybrid search and RRF

```
RRF(d) = SUM 1 / (k + rank_i(d))
```

```
k = 60
```

- No score calibration needed
- Robust: strong in both lists wins
- Our code: TF-IDF ranking + embedding ranking

Doc	Keyword rank	Semantic rank	RRF
B	2	1	0.0325
C	5	2	0.0315
A	1	8	0.0311
D	3	none	0.0159

B wins: good in both lists. A is first for keywords but weak on meaning.

Grounded prompting

```
SYSTEM:
  Answer using ONLY the context passages.
  Cite passages like [1], [2].
  If the answer is not in the context,
  say you do not know.

USER:
  Context:
  [1] (cybersecurity.md#1) ...passage...
  [2] (agro.txt#0) ...passage...
  Question: What are the phases of
  incident response?
```

Constrain

only the provided context

Number sources

so answers can cite [n]

Allow "I do not know"

refuse instead of guessing

RAG vs fine-tuning vs long context

	RAG	Fine-tuning	Long context
Updating knowledge	Re-index in minutes	Retrain	Paste at query time
Cost to update	Low	High	Tokens on every query
Citations	Natural: source chunks	Hard	Possible, no retrieval control
Best for	Large, changing private corpora	Style, format, task behaviour	Small corpora, one-off analysis
Main limit	Retrieval quality caps answers	Does not reliably add facts	Cost, limits, "lost in the middle"

RAG vs SLM: not an either/or

RAG

An architecture. Adds external, updatable knowledge at query time. Changes what the model knows.

SLM (small language model)

A model size class, about 0.5 to 8B parameters. Runs locally and cheaply. Changes what the model costs.

Real question: LLM, SLM, LLM + RAG or SLM + RAG? An SLM brings the cost down; RAG brings the knowledge in.

Head-to-head: which is better?

Dimension	Large LLM + RAG	SLM alone (fine-tuned)	SLM + RAG
Fresh / private facts	Strong	Weak: frozen in weights	Strong
Reasoning and writing	Best	Limited, task-specific	Moderate
Cost and latency	Highest	Lowest	Low
Privacy / offline	Cloud API	Fully local	Fully local
Hallucination control	Good with citations	Hard to audit	Fair, verify citations
Setup effort	Medium	Training data needed	Medium

General tendencies, not benchmarks. Test on your own data.

Choosing: a decision guide

Changing or large private corpus, citations needed	→ RAG
Narrow, stable task: classify, extract, fixed tone	→ Fine-tuned SLM
Offline, private or low compute	→ SLM + RAG
Multi-step reasoning across many documents	→ Large LLM + RAG
High volume, latency-critical, simple questions	→ SLM (+ cache)

Our lab runs both: Claude + RAG, or Llama 3.2 (3B) + RAG via Ollama.

Advanced RAG techniques

Query rewriting

Clarify or expand the question

HyDE

Embed a hypothetical answer

Reranking

Cross-encoder rescoring of top 50

Multi-hop

Retrieve, reason, retrieve again

Graph RAG

Entity graph for global questions

Agentic RAG

Agent decides when and what to fetch

Evaluating RAG

Retrieval

Recall@k: is a relevant chunk in the top k?

MRR: 1 / rank of the first relevant chunk

nDCG: rewards relevant chunks ranked higher

Needs: questions labelled with the right chunks

Generation

Faithfulness: claims supported by the context

Answer relevance: does it address the question?

Context precision: how much retrieved text was useful

Judging: LLM-as-judge plus human spot checks

Evaluate retrieval first: if the right chunk is missing, generation cannot fix it

Failure modes and security

Failure	Typical cause	Mitigation
Right chunk not retrieved	Vocabulary mismatch, poor chunks	Hybrid search, tune chunking, rerank
Answer ignores context	Weak prompt or small model	Stricter prompt, citations, fewer passages
Invented citations	Model makes up [n]	Check citations map to retrieved chunks
Prompt injection in documents	Retrieved text contains instructions	Treat docs as data, least-privilege tools
Data leakage	Private text sent to an API	Local model, redaction, access control
Unsafe index file	Pickle runs code on load	Only load your own; use safe formats

C

30 min

Hands-on lab

Run the repo: First-simple-rag

- Clone, install and run the repo
- Read the code: chunking, retrieval, generation
- Live demo, experiments and exercises

Our repo: First-simple-rag

github.com/sksdbest/First-simple-rag

```
First-simple-rag/
|-- rag.py                whole pipeline
|-- docs/
|   |-- agro.txt          farming sample
|   `-- cybersecurity.md  security sample
|-- tests/test_rag.py     4 unit tests
|-- requirements.txt
|-- README.md
`-- LICENSE
```

Stage	Function (line)
Load	`read_file()  :53`
Chunk	`chunk_text()  :60`
Index	`Index.__init__()  :84`
Retrieve	`Index.search()  :108`
Generate	`answer()  :185`

Setup: clone and install

1 **Get the code**

```
git clone https://github.com/sksdbest/First-simple-rag.git
cd First-simple-rag
```

2 **Install**

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

3 **Smoke test**

```
python rag.py ingest docs/
# Indexed 4 chunks from 2 files (tf-idf).
```

Windows: activate with `venv\Scripts\activate`

Choose a generator

Claude

Best answers. Needs an API key and credit.

```
export
ANTHROPIC_API_KEY=
  "sk-ant-..."
```

Ollama (local SLM)

Free, private, runs on your laptop.

```
ollama pull llama3.2
```

Extractive

Nothing to set up. Shows matching passages.

```
(default when neither
 is available)
```

Same retrieval in all three. Only the last step changes.

What is inside docs/

agro.txt

One paragraph about wheat: when it is sown and harvested, soil and irrigation.

agro.txt#0

cybersecurity.md

- # Zero Trust Basics
- # Incident Response
- # Phishing Defense

md#0 md#1 md#2

2 files -> 4 chunks. Add your own .txt, .md or .pdf and re-run ingest.

Code walk 1: chunking

```
def chunk_text(text, size=180, overlap=40):
    out = []
    for section in re.split(r"\n(?=#{1,6} )|\n\s*\n", text):
        words = section.split()
        if not words:
            continue
        step = max(1, size - overlap)
        out += [" ".join(words[i:i + size])
                for i in range(0, len(words), step)]
    return out
```

Split on headings and blank lines

Window of 180 words

Step 140 = 40 words of overlap

In the repo: rag.py, line 60

Code walk 2: retrieval and RRF

```
sims = (tfidf.transform([query]) @ tfidf_mat.T).toarray()[0]
rankings = [np.argsort(-sims)]            # keyword ranking
if dense is not None:
    q = embed([query])[0]
    rankings.append(np.argsort(-(dense @ q)))   # semantic
scores = {}
for r in rankings:
    for rank, i in enumerate(r[:50]):
        scores[i] = scores.get(i, 0) + 1.0 / (60 + rank)
top = sorted(scores, key=scores.get, reverse=True)[:k]
```

Keyword ranking (TF-IDF cosine)

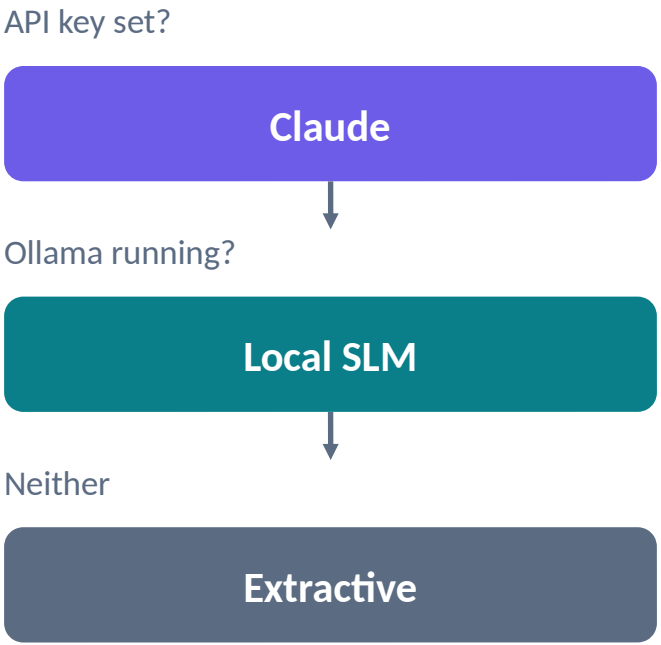
Semantic ranking (if installed)

Fuse with RRF, k = 60

In the repo: rag.py, line 108 (Index.search), simplified

Code walk 3: generation backends

```
def pick_backend():
    b = os.environ.get("RAG_BACKEND", "auto")
    if b != "auto":
        return b
    if os.environ.get("ANTHROPIC_API_KEY"):
        return "anthropic"
    return ("ollama" if ollama_up()
            else "extractive")
```



In the repo: rag.py, line 176

Demo 1: index and ask

```
$ python rag.py ingest docs/
Indexed 4 chunks from 2 files (tf-idf).

$ python rag.py ask "What are the phases of incident response?" -k 2
(extractive mode ...)
[1] (cybersecurity.md#1)
# Incident Response The incident response lifecycle has six phases:
preparation, identification, containment, eradication, recovery, ...

Sources:
  [1] cybersecurity.md (chunk 1)
```

4 chunks = 1 (agro) + 3 (security sections)

Top hit is the right chunk

With a key or Ollama: a written answer citing [1]

Demo 2: safe failure and tests

```
$ python rag.py ask "quantum chromodynamics"
I couldn't find anything relevant in the documents.

$ pip install pytest
$ pytest -q
....                                                    [100%]
4 passed in 0.91s
```

Out-of-scope question: no invented answer

pytest is not in requirements.txt: install it

4 tests: chunking, retrieval, no-hit, backend

Demo 3: where keywords fail

```
$ python rag.py ask "How do I stop credential theft emails?" -k 1
[1] (cybersecurity.md#1)          <-- wrong chunk
# Incident Response ... isolated to stop lateral movement ...

$ python rag.py ask "protecting accounts with hardware tokens"
I couldn't find anything relevant in the documents.
```

Why?

Keyword search needs shared words. Query 1 shares only "stop" with the wrong chunk. The Phishing Defense chunk is about the right topic, but uses other words (phishing, MFA, FIDO2).

Demo 4: add meaning, swap models

#	Command	What to watch
1	`pip install sentence-transformers`	Large download (PyTorch): do it before class
2	`python rag.py ingest docs/`	Message now says (hybrid), not (tf-idf)
3	`re-run the 2 failing queries`	Predict first: does Phishing Defense rank first?
4	`RAG_BACKEND=ollama python rag.py ask "..."`	Local SLM answer
5	`RAG_BACKEND=anthropic python rag.py ask "..."`	Compare citations, quality, speed

Experiments: write a hypothesis first

Change	How	Predict
Chunk size 60 vs 400 words	Edit CHUNK_WORDS (line 36), re-ingest	Precision vs context
Overlap 0 vs 40	Edit CHUNK_OVERLAP (line 37), re-ingest	Answers cut at boundaries?
Top-k 1 vs 4	Use the -k flag, or TOP_K (line 38)	Recall vs noise
Generator: Claude vs local SLM	RAG_BACKEND=anthropic / ollama	Citations, speed
Prompt injection	Add "ignore previous instructions" to a doc, re-ingest	Does the answer obey it?

Lab exercises

Easy	Fork the repo and add your own course notes to docs/
Medium	Add a .docx loader using python-docx
Medium	Write an eval script: Recall@k on 10 labelled questions
Hard	Add a cross-encoder reranker and compare results
Hard	Replace the pickle index with Chroma or FAISS

Easy = in class Medium = take-home Hard = mini-project

D

20 min

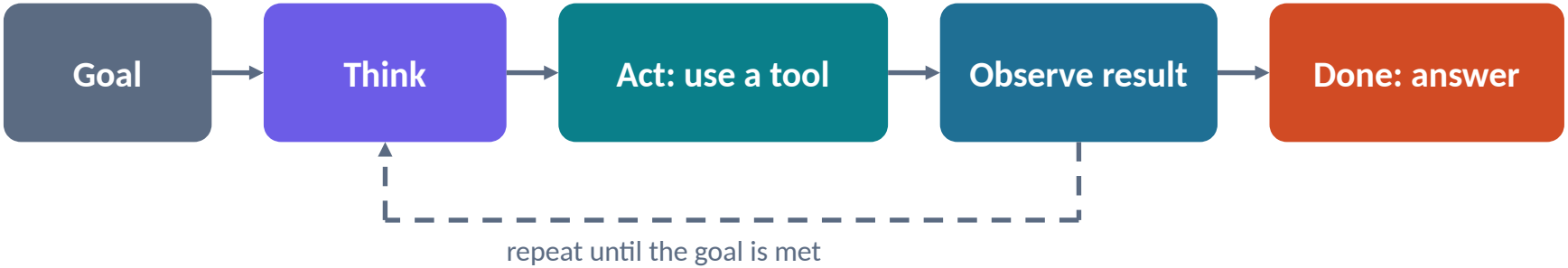
Agents and memory

Knowledge integration, memory architectures, compression

- What an AI agent is
- Agent knowledge integration
- Agent memory architectures and patterns
- Knowledge compression

What is an AI agent?

An agent is an LLM that decides what to do next, uses tools, and repeats until the goal is met.



LLM

The brain: reasons and decides

Tools

Search, SQL, code, APIs

Memory

What it has learned so far

Loop

Keeps going until done

Chatbot vs RAG vs agent

LLM chatbot

One step: answers from what it learned in training.

e.g. "What is cosine similarity?"

RAG

Fixed steps: our code always retrieves, then the LLM answers.

e.g. "Answer from our PDFs, with citations"

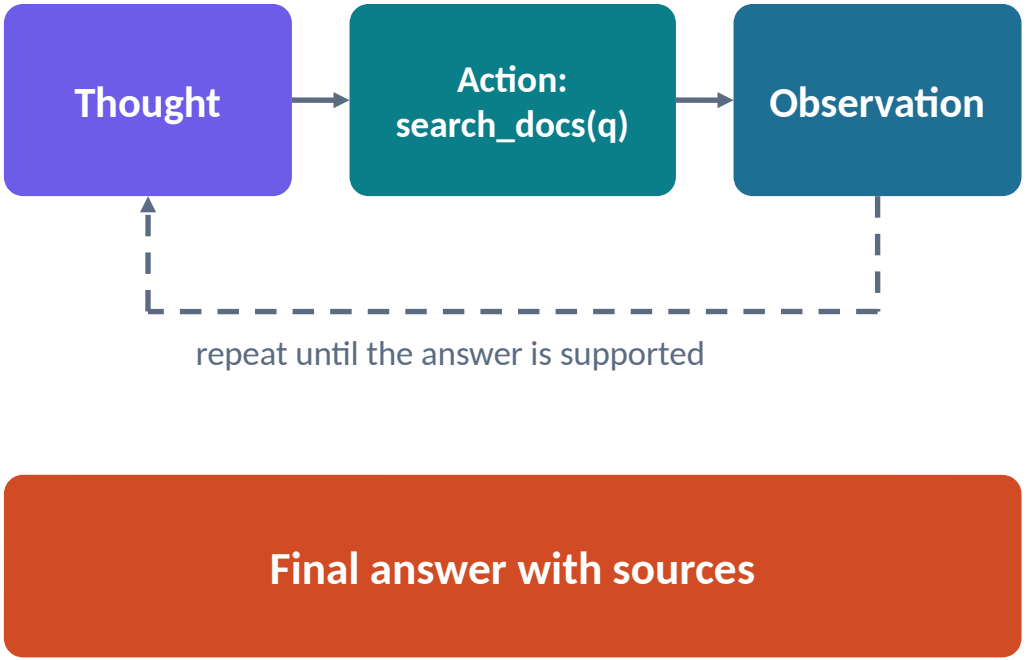
Agent

Flexible steps: the LLM picks the tools and the order, and loops.

e.g. "Find similar past incidents, check logs, draft a report"

RAG is often used as one tool inside an agent

Agent knowledge integration



RAG as a tool: the agent decides when to retrieve

Many sources: documents, SQL, web are all tools

Multi-hop: retrieve, reason, retrieve again

Guardrails: tool permissions, source tracking

Agent memory architectures



Working

Current task state in the context window

Like thinking out loud



Episodic

Past interactions with timestamps. Vector store plus time

Like remembering last week



Semantic

Facts and knowledge. RAG over documents or a graph

Like knowing facts



Procedural

How-to: skills, prompts, tool recipes

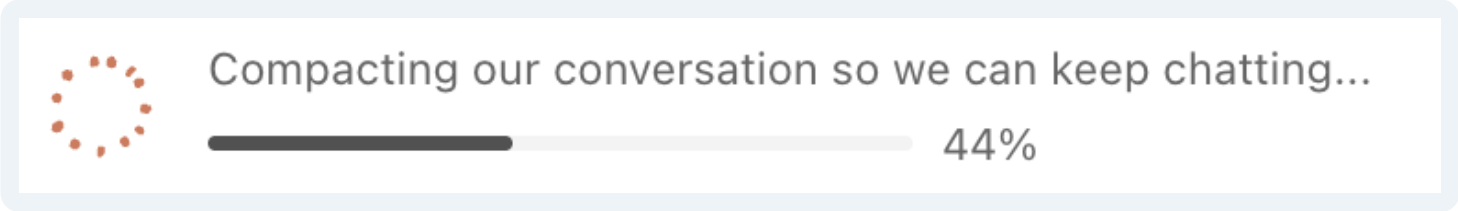
Like riding a bike

Short-term lives in the context window. Long-term lives in external stores, retrieved on demand.

Memory design patterns

Pattern	How it works	Good for	Watch out
Sliding buffer	Keep the last N turns	Simple chat	Forgets early facts
Summary memory	LLM summarises old turns	Long chats	Lossy, can drift
Vector memory	Embed and retrieve past notes	Personalisation	Irrelevant recalls
Tiered (MemGPT)	Main context + archive, paged by agent	Long-running agents	Complexity
Entity / graph	Facts as nodes and edges	Relationships	Extraction errors

Why do we need knowledge compression?



A real agent hitting its limit: the history is compacted so work can continue

Context is finite

The model reads a fixed token window. Long chats, tool outputs and retrieved text fill it.

Tokens cost money and time

More context means slower and costlier calls, and the middle gets ignored.

So we compress

Keep the meaning, drop the bulk, then carry on.

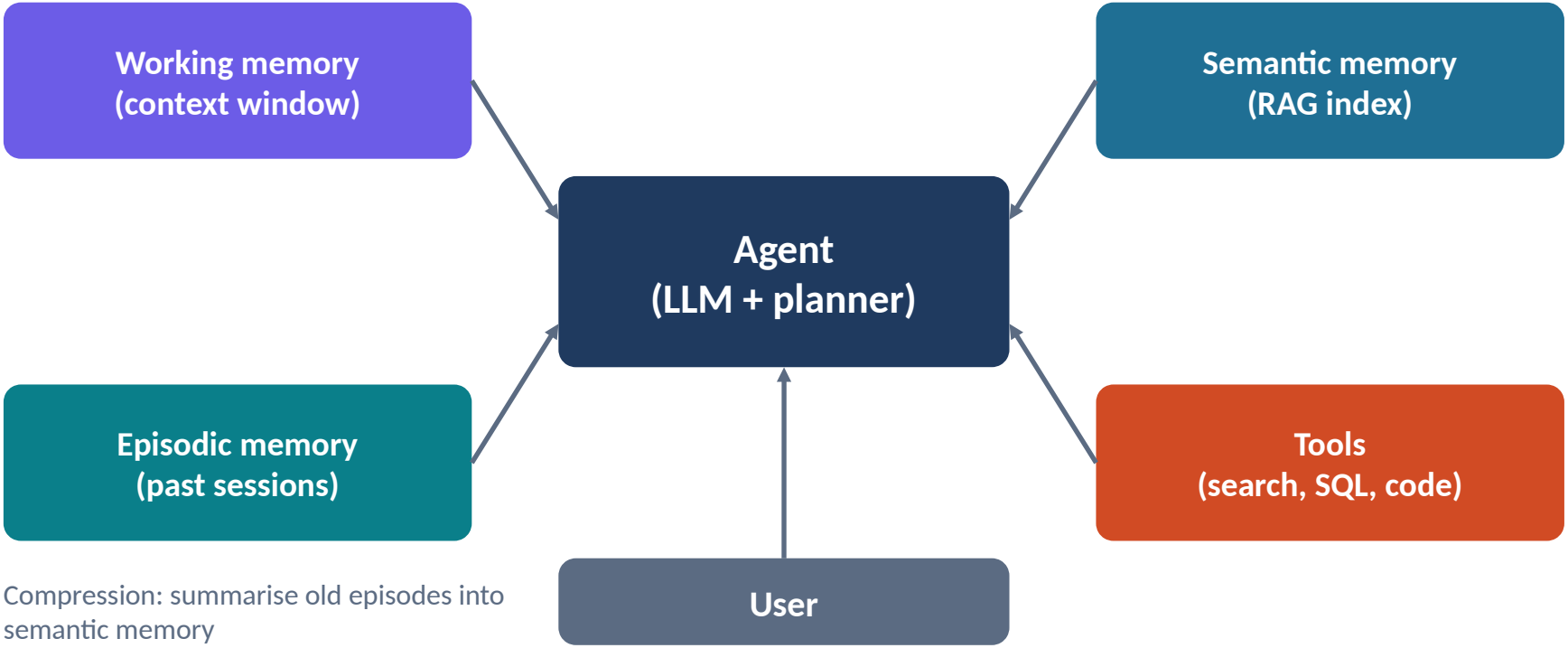
Knowledge compression

- Summaries and hierarchical summaries
- Context pruning and reranking
- Prompt compression (e.g. LLMLingua)
- Embedding quantisation, fewer dimensions
- Distil knowledge into a smaller model

1,536 B	float32, 384 dims
384 B	int8: 4x smaller
48 B	binary: 32x smaller

Trade-off: smaller is cheaper and faster, but lossy. Re-measure recall.

Putting it together



Compression: summarise old episodes into semantic memory

Key takeaways

1. Retrieval grounds LLMs in fresh, private evidence
2. Embeddings turn meaning into geometry; hybrid search adds exact matching
3. Chunking and retrieval quality matter as much as model size
4. RAG changes what a model knows; an SLM changes what it costs. Combine them
5. Agents need layered memory plus compression, and documents are untrusted input

Mini-project ideas

Course-notes tutor

RAG over your PDFs with citations

Incident lookup

Retrieve similar past security incidents

Agent with memory

A chatbot that recalls preferences across sessions

Benchmark study

Chunk size x retriever x k on 20 labelled questions

Deliverable: GitHub repo + 2-page report with Recall@k and faithfulness

References and further reading

- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain QA. EMNLP.
- Reimers & Gurevych (2019). Sentence-BERT. EMNLP.
- Robertson & Zaragoza (2009). The Probabilistic Relevance Framework: BM25 and Beyond.
- Cormack et al. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. SIGIR.
- Malkov & Yashunin (2018). Efficient and robust approximate nearest neighbor search using HNSW graphs. TPAMI.
- Johnson et al. (2019). Billion-scale similarity search with GPUs (FAISS). IEEE Trans. Big Data.
- Yao et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models.
- Packer et al. (2023). MemGPT: Towards LLMs as Operating Systems.
- Liu et al. (2023). Lost in the Middle: How Language Models Use Long Contexts.
- Es et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation.
- Lab code: your GitHub repository (simple-rag).



Thank you

Questions and discussion

Questions?

Code
github.com/sksdbest/First-simple-rag

Take-home
exercises and mini-project

Contact
skumars@iitbhilai.ac.in
goodhacker.in